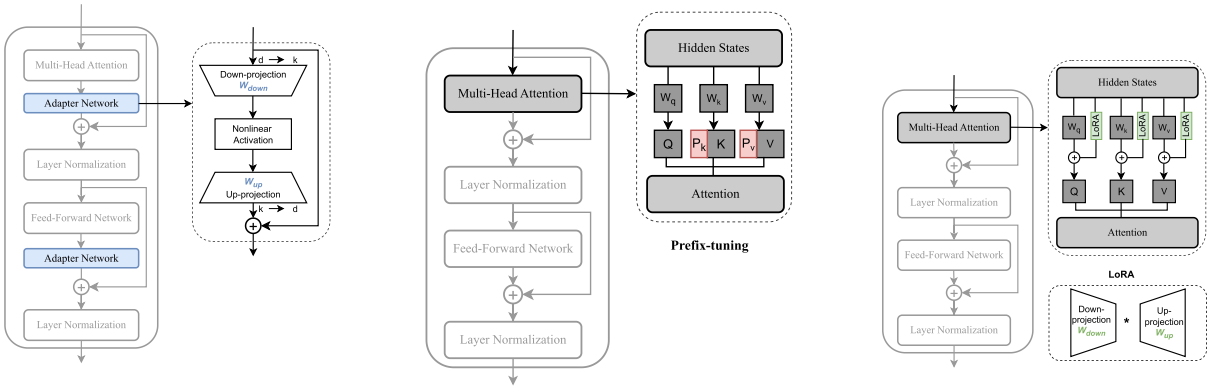


(a) Sequential Adapter (b) Prefix-tuning (c) LoRA

Fig. 3. Architecture of three representative PEFT methods: (a) Sequential Adapter, (b) Prefix-tuning, and (c) LoRA. Sequential Adapter introduces lightweight adapter networks into the Transformer layers, using down-projection (W_{down}) and up-projection (W_{up}) operations with a nonlinear activation. Prefix-tuning prepends trainable prefix vectors to the hidden states at each layer, modifying the input to the attention mechanism. LoRA applies low-rank decomposition to specific weight matrices (e.g., Query (Q), Key (K), and Value (V)) in the attention mechanism, with down-projection (W_{down}) and up-projection (W_{down}) layers operating in parallel to the frozen pretrained weights. These methods represent a broad range of PEFT strategies aimed at reducing the number of trainable parameters while preserving model performance, forming the foundation for numerous subsequent PEFT methods.

their weights to further reduce inference costs. Akin to the sequential adapter, the Tiny-Attn Adapter is injected right after the multi-head attention layer.

AdapterFusion [20] integrates multiple task-specific adapters into a single module, allowing for effective knowledge transfer across related tasks without modifying the original pretrained model. It provides a practical and efficient approach to task composition, enabling the transferability of pretrained models across multiple tasks while minimizing the computational costs associated with model fine-tuning. However, AdapterFusion requires additional trainable parameters in the composition layers, which increases the computational costs. **MerA** (Merging Pretrained Adapters) [21] addresses this limitation by merging pretrained adapter parameters through summation and averaging strategies, avoiding the introduction of other extra trainable parameters. Employing the optimal transport method [80], [81] to align adapter weights and activations, MerA achieves better performance with fewer trainable parameters compared to AdapterFusion. **Hyperformer++** [22] utilizes a shared hypernetwork [82] to learn task-specific and layer-specific adapter parameters conditioned on task and layer ID embeddings. By sharing knowledge across tasks via hypernetworks and enabling the model to adapt to each task through task-specific adapters, significantly reducing the number of trainable parameters. **AdapterSoup** [23] is developed to address cross-domain task adaptation, which first trains multiple adapters across various domains and then employs domain clustering [83] to select the most appropriate top-k adapters for the new domain. Fine-tuning parameters for the new domain in AdapterSoup are determined by calculating the weighted average of the selected k adapters. Additionally, AdapterSoup can strengthen in-domain results via weight averaging of adapters trained on the same domain but with different hyperparameters.

2) Soft Prompt-Based Fine-Tuning: Soft prompt fine-tuning is a class of methods in which trainable continuous vectors, known as soft prompts, are inserted into the input or hidden state of the model. Unlike manually designed hard prompts, soft prompts are generated by searching for prompts in a discrete token space based on task-specific training data.

WARP (Word-level Adversarial ReProgramming) [24] appends special prompt tokens $[P_1], [P_2], \ldots, [P_l]$ and a [Mask] token before or after the sentences, depending on the prompt template. The training objective is to minimize the cross-entropy loss between the output of MLM and the verbalizer tokens $[V_1], [V_2], \ldots, [V_c]$ for classes $\{1, 2, \ldots, c\}$. Only the parameters of $[P_1], [P_2], \ldots, [P_l]$ and $[V_1], [V_2], \ldots, [V_c]$ are trainable, significantly reducing the number of trainable parameters. **Prompt-tuning** [25] incorporates additional l learnable prompt tokens, $P = [P_1], [P_2], \ldots, [P_l]$, into the model input $X \in \mathbb{R}^{n \times d}$ and then concatenates them to generate the final input $\hat{X}$, as defined in (7). During fine-tuning, only the prompt parameters of P are updated via gradient descent, while pretrained parameters remain frozen. The parameter cost of prompt-tuning is determined by the prompt length and token embedding dimension.

$$\hat{X} = \text{Concat}(P, X) = [P, X] \in \mathbb{R}^{(l+n) \times d}. \tag{7}$$

Prefix-tuning [11] proposes to prepend soft prompts $P = [P_1], [P_2], \ldots, [P_l]$ (l denotes the prefix length) to the hidden states of the multi-head attention layer. To stabilize training, an FFN is used to parameterize the soft prompts, as directly optimizing their values can cause instability. Two sets of prefix vectors, $\hat{P}_k$ and $\hat{P}_v$, are concatenated with the original key (K) and value (V) vectors of the attention layer. The self-attention mechanism with prefix-tuning is represented by (8) and 9. During training, only $\hat{P}_k$, $\hat{P}_v$, and the FFN parameters are optimized, while all other parameters of PLMs remain frozen. After training, the FFN is discarded, and only P_k and P_v are used for inference. The structure of prefix-tuning is shown in Fig. 3(b). **P-tuning** [26] also leverages soft prompts, $[P_1], \ldots, [P_i], [P_{i+1}], \ldots, [P_l]$, but differs in its approach. It concatenates these prompts to form a template, which is mapped